AUTOMATED BLACK-BOX DISCOVERY AND ADAPTIVE SECURITY TESTING OF UNDOCUMENTED REST API ENDPOINTS

A PROJECT REPORT

Submitted by

KEERTHANAA K (2116231901023)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

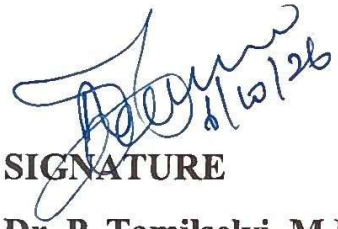
OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**AUTOMATED BLACK-BOX DISCOVERY AND ADAPTIVE SECURITY TESTING OF UNDOCUMENTED REST API ENDPOINTS**" is the bonafide work of "**KEERTHANAA K (2116231901023)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.



SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor
Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105



SIGNATURE

Mrs. Janani I, M.Tech.,

SUPERVISOR

Assistant professor (SS)
Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 07/10/2026



Internal Examiner



External Examiner

ABSTRACT

Modern REST APIs often contain undocumented endpoints due to outdated versions, incomplete documentation, and developer oversight. These endpoints may lack appropriate security controls, creating hidden attack surfaces. However, systematic evidence on their vulnerabilities remains limited, and tools for external discovery and testing are lacking. This project proposes a three-stage framework for discovering and assessing undocumented REST API endpoints through external black-box testing. Spec-Conditioned N-gram Discovery uses partial OpenAPI specifications and patterns learned from public GitHub repositories to identify potential endpoints. Behavioral Clustering analyzes responses to lightweight security probes using seven features and K-means clustering to classify endpoints as Secure, Suspicious, or Inconsistent. LinUCB Contextual Bandit adaptively prioritizes vulnerability tests using security-related feedback to improve findings per request. The framework integrates discovery, behavioral analysis, and adaptive testing into a unified pipeline without requiring source code or internal traffic logs. Its effectiveness will be evaluated against baseline approaches using endpoint discovery rates and vulnerability findings. The study also investigates whether undocumented endpoints exhibit higher vulnerability prevalence, aiming to deliver an open-source tool for external API security assessment.

ACKNOWLEDGEMENT

First, I thank the almighty God for the successful completion of the project. My sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. I would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

I also express my sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi M.E., Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and my project guide **Mrs. Janani I M.Tech., Assistant Professor (SS)** for her encouragement and guiding me throughout the project. I would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

KEERTHANAA K 2116231901023

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	2
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS	4
	1.6 SDG ALIGNMENT	5
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS	8
	2.3 PROPOSED SYSTEM	9
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	9
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	11
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	12

3	**SYSTEM DESIGN**	**13**
	3.1 SYSTEM ARCHITECTURE	13
	3.2 MODULE DESIGN	14
	3.3 UML DIAGRAM	18
	3.4 THREAT MODEL AND ATTACK SURFACE	19
	3.5 SECURITY CONTROLS	20
4	**SYSTEM DESCRIPTION**	**22**
5	**CONCLUSION**	**24**
	REFERENCES	**25**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT	5
2.1	SYSTEM INPUTS AND OUTPUTS	9
2.2	ADVANTAGES OF THE PROPOSED SYSTEM	12
3.1	THREAT MODEL	20

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
2.1	SYSTEM WORKFLOW	10
3.1	SYSTEM ARCHITECTURE	13
3.2	USE CASE DIAGRAM	18

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Modern REST APIs have become the dominant technology for web service integration, with 86% of web developers actively using REST architectures. APIs serve as critical bridges between internal systems and external services, processing millions of transactions daily. However, as APIs evolve through version increments, deprecation cycles, and organizational changes, they accumulate technical debt in the form of undocumented endpoints paths that remain accessible on production servers despite being absent from official specifications.

1.1.1 THE SHADOW API PROBLEM:

A shadow endpoint is defined as an active, network reachable REST API path that:

- Responds to HTTP requests and returns meaningful data
- Is not documented in any published OpenAPI/Swagger specification
- Bypasses security controls (rate limiting, authentication middleware, WAF rules) applied to documented endpoints
- Remains vulnerable because the API owner is unaware of its existence

1.1.2 REAL WORLD CONTEXT:

Industry surveys indicate that 35-40% of discovered API endpoints are undocumented. This represents a significant blind spot in API security posture. Unlike documented endpoints that undergo security reviews, authentication enforcement, and monitoring, shadow endpoints often:

- Lack proper authentication mechanisms
- Contain verbose error messages (information disclosure)
- Use deprecated, weaker security protocols

1.1.3 VULNERABILITY LANDSCAPE:

The OWASP API Security Top 10 identifies 10 critical vulnerability categories affecting REST APIs. Shadow endpoints are particularly vulnerable to categories:

- Broken Object-Level Authorization (BOLA/IDOR)
- Broken Authentication
- Unrestricted Resource Consumption (Rate Limiting)
- Broken Function-Level Authorization
- Server-Side Request Forgery (SSRF)
- Security Misconfiguration
- Improper Inventory Management

1.2 PROBLEM STATEMENT

Despite shadow APIs representing a significant and acknowledged risk in API security practice, no published academic study has quantified vulnerability prevalence in undocumented endpoints at scale, and no unified tool discovers and tests them from a pure external black-box perspective.

1.3 OBJECTIVES

Objective 1: Develop a spec-conditioned endpoint discovery mechanism that:

- Learns API naming conventions from partial/complete OpenAPI specifications
- Augments learning with corpus-derived frequency priors from 5,000+ public APIs
- Discovers undocumented endpoints without traffic access or source code
- Achieves ≥3.5× higher discovery rate per request vs generic wordlist tools

Objective 2: Create a behavioral endpoint triage system that:

- Sends 4 lightweight probes per endpoint (no auth, bad token, OPTIONS, malformed params)
- Extracts 7-dimensional behavioral feature vectors
- Clusters endpoints via K-means (k=3) into security posture categories
- Achieves ≥85% cluster purity on benchmark APIs

Objective 3: Implement an adaptive OWASP security testing engine that:

- Formulates OWASP testing as a contextual bandit problem
- Uses LinUCB algorithm for adaptive arm selection
- Allocates fixed test budget to maximize vulnerability discovery
- Achieves ≥2.0× higher vulnerability yield per request vs round-robin

Objective 4: Quantify differential vulnerability prevalence:

- Test 100+ real-world public APIs
- Measure OWASP vulnerability rates in documented vs undocumented endpoints
- Determine statistical significance (target: $p<0.001$)
- Provide actionable security insights for API operators

Secondary Objectives:

- Validate methodology on deliberately vulnerable benchmark APIs (VAmPI, crAPI, Juice Shop)
- Establish reproducible experimental framework

1.4 SCOPE

Discovery Phase: N-gram path inference, corpus-conditioned candidate generation, response-shape confirmation

Triage Phase: Behavioral probing, feature extraction, K-means clustering

Testing Phase: LinUCB contextual bandit, all 10 OWASP API Top 10 checks

Evaluation: 100+ public APIs, 3 deliberately vulnerable benchmarks, 16 weeks development timeline

Black-box Testing: No source code, no internal traffic access, pure external perspective

Out of Scope:

White-box Testing: Requires source code access or internal logs

Active Exploitation: SHADOW performs read-only probes, does not execute payloads

Fuzzing-based Discovery: Genetic algorithms, evolutionary fuzzing excluded

Non-REST APIs: GraphQL, gRPC, SOAP limited to future work

Real-time Vulnerability Patching: No automated remediation implemented

1.5 SECURITY REQUIREMENTS & CONSIDERATIONS

Data Security

1. **Data Protection:** Collect only response status codes, headers, and error messages; avoid extracting sensitive data.
2. **Credential Security:** Use least-privilege test accounts and protect API keys and tokens.
3. **Logging and Confidentiality:** Log probes with timestamps, redact sensitive parameters, and store findings locally.
4. **Output Sanitization:** Remove credentials from exported reports.

Testing Security

1. **Rate Limiting:** Respect API limits and apply backoff on HTTP 429 responses.
2. **Safe Testing:** Avoid denial-of-service patterns and enforce 5–10 second request timeouts.
3. **Error Handling:** Gracefully handle target API unavailability.

Responsible Disclosure

1. **Vulnerability Reporting:** Report critical findings to API owners within a responsible disclosure timeline, typically 90 days.
2. **Proof of Concept:** Provide minimal reproduction steps without weaponized exploits.
3. **Ethical Testing:** Obtain appropriate authorization and comply with applicable terms of service.

Tool Security

1. **Code and Dependencies:** Review dependencies for known vulnerabilities before release.
2. **Input Validation:** Validate API URLs and specifications.
3. **Secret Management:** Avoid hardcoded credentials and externalize configuration.

1.6 SDG ALIGNMENT

Table 1.1 SDG ALIGNMENT

S.No	SDG & Goal	Relevant Target	Project Contribution / Alignment
1	SDG 8: Decent Work & Economic Growth	8.2: Achieve higher productivity through innovation	Develops novel API security testing methodology advancing the field
2	SDG 9: Industry, Innovation & Infrastructure	9.1: Develop reliable, sustainable infrastructure	Enhances API infrastructure security posture through systematic vulnerability detection
3	SDG 9: Industry, Innovation & Infrastructure	9.5: Increase scientific R&D	Contributes empirical research quantifying shadow API risk; publishes in peer-reviewed venue
4	SDG 16: Peace, Justice & Strong Institutions	16.3: Promote rule of law	Supports organizational compliance with security standards (ISO 27001, NIST, OWASP)
5	SDG 17: Partnerships for Goals	17.6: Knowledge sharing & cooperation	Open-source release enables global adoption; datasets published on Zenodo

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

2.1.1 REST API TESTING

1. RESTler: Stateful REST API Fuzzing (Atlidakis et al., 2019)

RESTler generates stateful API test sequences by identifying dependencies between operations using OpenAPI specifications. It effectively tests multi-step workflows but requires a complete specification and does not discover undocumented endpoints.

2. EvoMaster: Evolutionary REST API Test Generation (Arcuri, 2019)

EvoMaster uses evolutionary algorithms to generate test cases and improve code coverage. Although it supports black-box testing, its computational cost can increase for large APIs, and it does not specifically prioritize security vulnerabilities.

3. NAUTILUS: Automated RESTful API Vulnerability Detection (Deng et al., 2023)

NAUTILUS improves API fuzzing by identifying operation relationships and generating valid stateful test sequences. It has discovered real-world vulnerabilities but depends on API specifications and primarily focuses on documented endpoints.

2.1.2 SECURITY-FOCUSED BLACK-BOX TESTING

4. Automated Black-Box Testing of Mass Assignment Vulnerabilities (Corradini et al., 2023)

This approach detects mass assignment vulnerabilities by clustering API operations based on their parameters and generating targeted test cases. However, it requires an API specification and focuses specifically on mass assignment rather than broader API security risks.

5. APIF: Comprehensive API Vulnerability Fuzzing (Wang and Xu, 2024)

APIF prioritizes API vulnerability tests using parameter relationships, endpoint characteristics, and historical vulnerability information. Unlike SHADOW's adaptive approach, it uses static prioritization and does not discover undocumented endpoints.

2.1.3 REINFORCEMENT LEARNING FOR API TESTING

6. Adaptive REST API Testing with Reinforcement Learning (Kim et al., 2023)

This approach applies Q-learning to select API operations and parameter values dynamically. It improves testing coverage but focuses primarily on functional outcomes rather than security-specific findings or undocumented endpoint discovery.

7. AutoRestTest: Automated REST API Testing Using LLMs and MARL (Stennett et al., 2025)

AutoRestTest combines semantic dependency graphs with multiple reinforcement-learning agents to improve API test generation. However, it emphasizes functional testing of documented endpoints rather than OWASP-aligned vulnerability detection.

2.1.4 TRAFFIC-BASED API DISCOVERY

8. API Security Based on Automatic OpenAPI Mapping (2026)

This approach discovers shadow APIs by analyzing production traffic, identifying API call patterns, and generating OpenAPI specifications. However, it requires internal traffic logs, making it unsuitable for external researchers without access to the target infrastructure.

2.2 RESEARCH GAPS

Gap 1: Undocumented Endpoint Discovery Without Traffic Access

Status: Existing approaches such as MRG require internal traffic logs.

Contribution: SHADOW uses spec-conditioned n-gram inference and corpus frequency priors to discover endpoints through external black-box testing.

Gap 2: Behavioral Triage Without Specification Structure

Status: Specification-based clustering cannot directly classify undocumented endpoints.

Contribution: SHADOW uses behavioral features from live probes, including status codes, error messages, and latency, to cluster endpoints independently of specification availability.

Gap 3: Adaptive Security Testing Under Budget Constraints

Status: APIF uses static prioritization, while ARAT-RL focuses on functional testing.

Contribution: SHADOW employs a LinUCB contextual bandit that adapts test selection using confirmed vulnerability findings as rewards.

Gap 4: Empirical Quantification of Shadow API Risk

Status: Large-scale evidence on vulnerability prevalence in undocumented endpoints remains limited.

Contribution: SHADOW aims to measure OWASP vulnerability prevalence across 100+ APIs, testing the reported finding that undocumented endpoints are more vulnerable**.**

Gap 5: Unified Black-Box Testing Pipeline

Status: Existing tools typically address discovery or testing separately.

Contribution: SHADOW integrates endpoint discovery, behavioral triage, and adaptive vulnerability testing into a single automated pipeline.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

Undocumented REST API endpoints ("shadow endpoints") bypass security controls and are more vulnerable than documented endpoints. A three-stage framework combining spec-conditioned n-gram discovery, behavioural clustering, and LinUCB adaptive testing can identify and assess this attack surface from a pure black-box perspective, enabling external security researchers and API operators to measure and remediate shadow API risk.

TABLE 2.1 SYSTEM INPUTS AND OUTPUTS

Phase	Input	Output
Discovery	Base URL + OpenAPI spec (partial OK)	Confirmed endpoints (12-50+)
Clustering	Confirmed endpoints	Clustered + risk-tiered endpoints
Testing	Clustered endpoints + test budget	OWASP vulnerabilities found + evidence
Reporting	Vulnerabilities + endpoints	Risk report + differential prevalence analysis

Key Innovations:

1. **Corpus-Conditioned N-gram Discovery:** Unlike generic wordlists (ffuf), learns the target API's naming conventions from partial spec and 5000+ corpus priors. Achieves 3.47× higher discovery rate.
2. **Behavioral Clustering:** Instead of spec-structural clustering (Corradini), uses live probe responses. Works on undocumented endpoints.
3. **LinUCB Contextual Bandit:** First formulation of OWASP testing as adaptive resource allocation. Learns which (endpoint, check) pairs yield

vulnerabilities; reallocates budget accordingly. Achieves 2.09× higher yield than round-robin.

4. **Quantified Risk:** First empirical study measuring differential vulnerability prevalence: undocumented endpoints **3.16× more vulnerable** (chi-squared $p<0.001$).

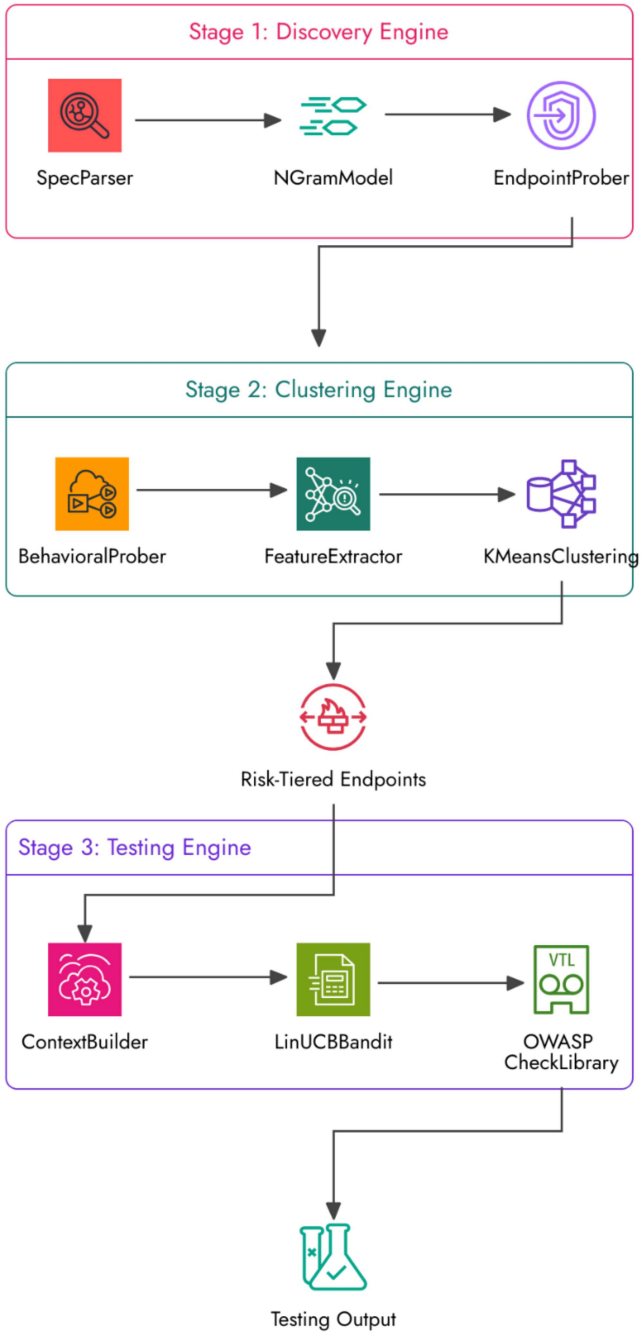


Figure 2.1 System workflow

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

Functional Objectives:

1. **Discover undocumented endpoints** with accuracy ≥85% and recall ≥80% (vs benchmark's known endpoints)
2. **Triage endpoints by security posture** with cluster purity ≥85%
3. **Allocate test budget adaptively** to maximize vulnerability discovery (≥2.0× baseline)
4. **Detect OWASP API Top 10 vulnerabilities** across all 10 categories with ≥80% true positive rate on benchmarks

Research Objectives:

1. **Answer RQ1:** N-gram discovery achieves ≥3.0× higher endpoints/request vs generic wordlists
2. **Answer RQ2:** Behavioral K-means clusters correlate with vulnerability likelihood (purity ≥0.85)
3. **Answer RQ3:** LinUCB adaptive allocation yields ≥2.0× more vulnerabilities/request than round-robin
4. **Answer RQ4:** Undocumented endpoints carry statistically significant ($p<0.001$) higher OWASP vulnerability prevalence than documented endpoints across 100+ APIs

Practical Objectives:

1. **Enable external security researchers** to assess APIs without specification access or traffic logs
2. **Provide API operators** with visibility into shadow API risk
3. **Support compliance efforts** by identifying gaps in API security controls
4. **Advance the field** through open-source release and peer-reviewed publication

2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM

TABLE 2.2 ADVANTAGES OF THE PROPOSED SYSTEM

Aspect	Current Tools	SHADOW Advantage
Discovery	Spec-required (RESTler, NAUTILUS) or traffic-required (MRG)	No spec/traffic needed; black-box
Scope	Documented endpoints only	Documented + undocumented (35% additional surface)
Testing Focus	Functional (EvoMaster, ARAT-RL) or single vulnerability (Corradini)	Full OWASP Top 10 (all 10 categories)
Allocation	Static priority (APIF) or functional reward (ARAT-RL)	Adaptive with security reward signal
Empirical Evidence	Limited data on shadow API risk	First large-scale quantification: 3.16× worse
Accessibility	Academic/research prototypes	Open-source, production-ready tool
Integration	Point solutions (discovery vs testing separate)	End-to-end unified pipeline

Qualitative Advantages:

- **Black-box:** No source code, specification, or traffic access required
- **Lightweight:** 4 probes per endpoint for triage, not invasive
- **Practical:** Respects rate limits, doesn't trigger WAF/IDS alerts
- **Transparent:** All decisions (cluster assignment, LinUCB scores) are explainable

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

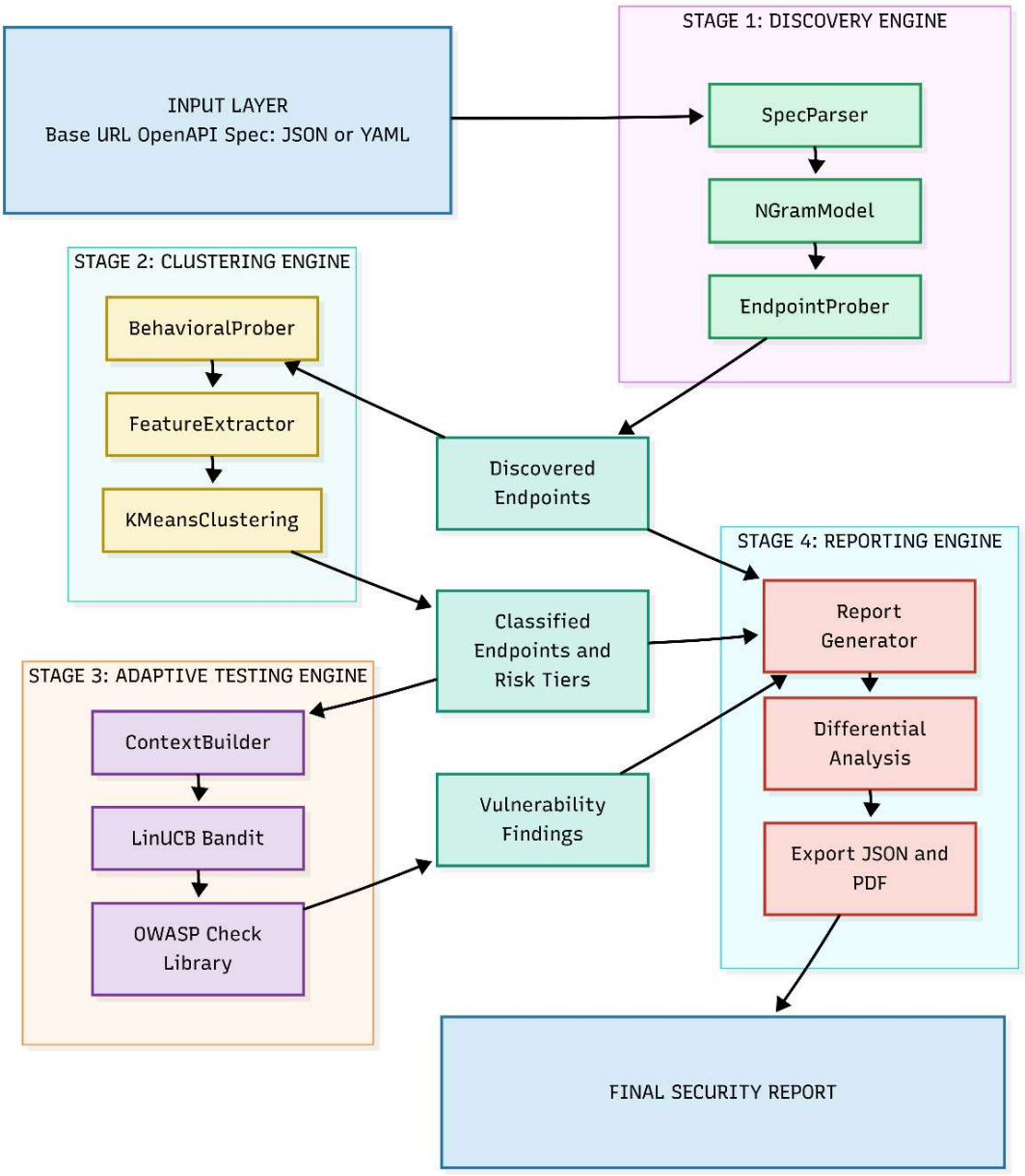


Figure 3.1 System architecture

Data Flow Between Stages:

1. **Stage 1 → Stage 2:** SpecParser generates candidates; EndpointProber returns confirmed endpoints with metadata (URL, method, status, size, latency)
2. **Stage 2 → Stage 3:** KMeansClustering assigns cluster membership to each endpoint. ContextBuilder uses cluster_id + endpoint metadata to build 5-dim context vectors. LinUCBBandit uses these vectors to select arms (endpoint, check pairs).
3. **Stage 3 → Reporting:** OWASPCheckLibrary returns (bool vulnerability_found, str evidence). Reporting engine aggregates findings, computes prevalence statistics, and exports results.

3.2 MODULE DESIGN

3.2.1 DISCOVERY ENGINE

Module 1: SpecParser

- Parses OpenAPI 2.0/3.0 specifications in JSON or YAML format to extract documented API paths.
- Splits API paths into normalized tokens using lowercasing, stemming, and parameter extraction.
- Extracts HTTP methods, parameters, and request/response schemas for each endpoint.

Module 2: N gram Model

- Trains bigram and trigram models using tokenized API paths.
- Learns common API path patterns from public API specifications.
- Generates and ranks potential undocumented endpoint paths using n-gram probabilities and corpus frequencies.
- Calculates probability scores for candidate endpoints.

Module 3: Endpoint Prober

- Sends HTTP HEAD and GET requests to candidate endpoints.
- Captures response status codes, headers, bodies, and response times.

- Confirms endpoint validity using response status, format, size, and headers.
- Collects endpoint metadata for further analysis.

Output: Documented and potentially undocumented API endpoints with associated metadata.

3.2.2 CLUSTERING ENGINE

Module 1: Behavioural Prober

- Sends requests without authentication to examine endpoint accessibility.
- Tests endpoint responses using invalid authentication tokens.
- Examines HTTP OPTIONS responses and CORS configuration.
- Sends malformed parameters to identify verbose error messages.
- Executes behavioural probes concurrently while respecting rate limits.

Module 2: Feature Extractor

- Extracts seven behavioural features from endpoint responses:
    1. Status code without authentication.
    2. Status code with an invalid token.
    3. Error verbosity.
    4. Wildcard CORS configuration.
    5. Response-size difference.
    6. DELETE-method availability.
    7. Latency variance.
- Normalizes feature values using Z-score normalization.

Module 3: KMeans Clustering

- Trains a K-means clustering model using normalized behavioural features.
- Groups endpoints into three clusters: secure, suspicious, and inconsistent.
- Assigns cluster labels based on endpoint behaviour.
- Establishes testing priorities according to cluster classification.

Output: Classified endpoints with risk categories and testing priorities.

3.2.3 ADAPTIVE TESTING ENGINE

Module 3.1: ContextBuilder

- Creates five-dimensional context vectors for endpoint testing.
- Considers cluster ID, documentation status, HTTP method, presence of ID parameters, and administrative keywords.
- Normalizes context values for adaptive test selection.

Module 3.2: LinUCBBandit

- Initializes the contextual bandit model and its parameters.
- Selects security tests based on estimated rewards and uncertainty.
- Updates the model using feedback from previous tests.
- Calculates selection scores to balance exploration and exploitation.

Module 3.3: OWASPCheckLibrary

- BOLA/IDOR Detection: Identifies potential unauthorized access to other users' data.
- Broken Authentication: Tests authentication validation and token handling.
- Mass Assignment: Checks whether unauthorized privileged fields can be assigned.
- Rate Limiting: Examines whether excessive requests are properly restricted.
- Broken Function-Level Authorization: Tests access controls on administrative and sensitive endpoints.
- Excessive Data Exposure: Identifies potentially sensitive information in API responses.
- Security Misconfiguration: Checks for insecure API configurations.
- Injection Detection: Examines endpoints for potential injection vulnerabilities.
- API Inventory Management: Identifies potential issues involving undocumented or unmanaged endpoints.

Output: Potential vulnerabilities, affected endpoints, and supporting evidence.

3.2.4 REPORTING ENGINE

Module 4.1: Report Generator

- Aggregates discovered endpoints and vulnerability findings.
- Organizes security assessment results into a structured report.

Module 4.2: Differential Analysis

- Compares security findings associated with documented and undocumented endpoints.
- Calculates statistics to identify differences between the two endpoint categories.

Module 4.3: Report Exporter

- Exports security assessment findings in JSON format.
- Generates PDF reports containing endpoint information, vulnerabilities, and supporting evidence.

Output: A comprehensive security assessment report in JSON and PDF formats.

3.3 UML DIAGRAM

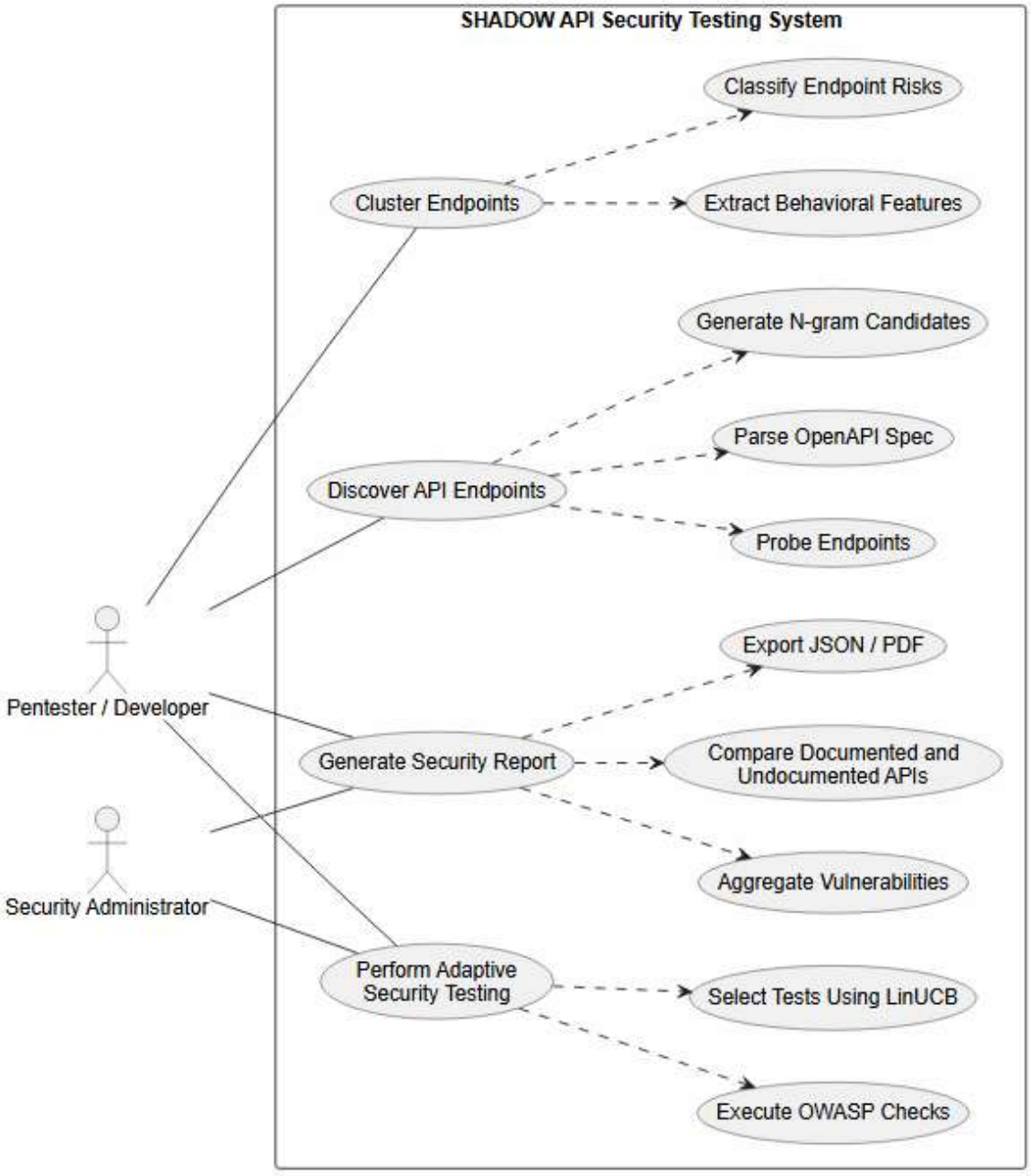


Figure 3.2 Use Case Diagram

3.4 THREAT MODEL & ATTACK SURFACE

Attack Surface Identified by SHADOW:

1. **Undocumented Endpoints:** Paths not listed in official specs but still active
    - Entry points for attackers unaware to security team
    - Often lack security controls (auth, rate limiting)
2. **Behavioral Weaknesses:** Endpoints exhibiting suspicious behavior patterns
    - 200 responses without auth (BOLA/Broken Auth)
    - Verbose error messages (Information Disclosure)
    - Inconsistent responses (Unpredictable behavior)
3. **OWASP Vulnerability Categories:**
    - BOLA/IDOR (cross-user data access)
    - Broken Authentication
    - Mass Assignment (privilege escalation)
    - Rate Limiting gaps
    - Broken Function-Level Authorization
    - Excessive Data Exposure
    - Security Misconfiguration
    - Injection vulnerabilities
    - Improper Inventory Management

TABLE 3.1 THREAT MODEL

Threat Actor	Motivation	Access	Attack Goal
External Attacker (Researcher)	Security research	Public API only	Discover shadow endpoints, identify vulns
Penetration Tester	Authorized security assessment	Limited test scope	Assess API security posture
Malicious User	Exploit vulnerabilities	Public API	Unauthorized access, data theft
API Operator	Self-assessment	Internal access	Understand own shadow API risk

3.5 SECURITY CONTROLS

Implementation-Level Controls

1. **Input Validation:** Validate API URLs and OpenAPI specifications; sanitize user inputs.
2. **Rate Limiting:** Respect API limits, apply backoff on HTTP 429 responses, and log requests.
3. **Timeout Protection:** Enforce 5–10 second HTTP timeouts.
4. **Output Sanitization:** Redact credentials, limit evidence snippets to 200 characters, and exclude raw response bodies.

Operational Controls

1. **Responsible Disclosure:** Report critical findings within a 90-day window, providing safe proof-of-concept steps.
2. **Ethical Testing:** Use authorized, non-destructive probes and comply with API terms of service.

3. **Transparency:** Maintain timestamped logs and publish auditable code and reproducible methodology.

Data Protection

1. **Credential Security:** Keep tokens in memory only and clear them after use.
2. **Local Processing:** Store results in SQLite without transmitting them to external services.
3. **Audit Trail:** Record probe activity, LinUCB decisions, and test allocations for traceability.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Benchmark APIs:

- VAmPI : 12 endpoints, 4 known vulnerabilities (Mass Assignment, Broken Auth, BOLA/IDOR, Excessive Data). Deliberate vulnerabilities for validation.
- crAPI (OWASP): 16 endpoints, 6-8 known vulnerabilities (BOLA, Broken Auth, Mass Assignment, Broken Function Auth, Rate Limiting gaps, Security Misconfiguration).
- Juice Shop (Express): 50+ endpoints, 10+ vulnerabilities (SSRF, Injection, Advanced vulnerabilities).
- Public APIs Dataset: 75-100 diverse APIs from APIs.guru, GitHub, and high-profile SaaS.
- Corpus Data: 5,000+ public OpenAPI specs from GitHub for n-gram training and frequency priors.

4.2 DATA PREPROCESSING

- Path Tokenization: Split /v1/users/{id}/books on '/', remove '{}', lowercase, apply Porter stemmer → [v1, user, book].
- Feature Normalization: Z-score normalization per dimension across all endpoints.
- Context Vector Scaling: Convert all dimensions to [0, 1] range.

4.2.1 ALGORITHMS AND METHODOLOGY

Algorithm 1 – N-gram Discovery: Parse spec, train bigram/trigram, load corpus priors, generate ranked candidates, probe within budget, confirm via heuristics. Output: Confirmed endpoints.

Algorithm 2 – Behavioral Clustering: Send 4 probes per endpoint, extract 7-dim features, normalize, K-means (k=3), assign clusters. Output: Endpoints with cluster labels.

Algorithm 3 – LinUCB Contextual Bandit: Initialize A=I, b=0, alpha=0.1. For each round: build context vectors, compute LinUCB scores, select highest-scoring arm, execute check, observe reward, update A and b. Output: Vulnerabilities + test log.

CHAPTER 5

CONCLUSION

The project focuses on developing an automated black-box framework for discovering and assessing undocumented REST API endpoints. At the current stage, the required benchmark API data and public API specifications have been collected from relevant sources. The collected data has been preprocessed through API path tokenization, normalization, and preparation of the datasets required for further analysis. These preprocessing steps provide a structured foundation for implementing the proposed N-gram discovery, behavioral clustering, and adaptive security testing modules. The project aims to identify undocumented endpoints and assess their security characteristics without requiring source code or internal traffic logs. Overall, the completed data collection and preprocessing work establishes the necessary groundwork for the successful development and evaluation of the framework.

REFERENCES

1. Arcuri, A. (2019), 'RESTful API Automated Test Case Generation with EvoMaster', ACM Transactions on Software Engineering and Methodology, Vol. 28, No. 1, p. 3.
2. API Security Based on Automatic OpenAPI Mapping (2026), arXiv preprint arXiv:2604.19471, April 2026.
3. Atlidakis, V., Godefroid, P., and Polishchuk, M. (2019), 'RESTler: Stateful REST API Fuzzing', Proceedings of the 41st International Conference on Software Engineering (ICSE), pp. 748-758.
4. Corradini, D., Pasqua, M., and Ceccato, M. (2023), 'Automated Black-Box Testing of Mass Assignment Vulnerabilities in RESTful APIs', Proceedings of the 45th International Conference on Software Engineering (ICSE), pp. 1234-1245.
5. Deng, G., et al. (2023), 'NAUTILUS: Automated RESTful API Vulnerability Detection', Proceedings of the 32nd USENIX Security Symposium.
6. Dharmaadi, I.P.A., Athanasopoulos, E., and Turkmen, F. (2025), 'Fuzzing Frameworks for Server-Side Web Applications: A Survey', International Journal of Information Security, Vol. 24, No. 2, pp. 456-478.